

UNIVERSITY OF MANAGEMENT AND TECHNOLOGY (UMT)

School of Systems and Technology

Department of Cyber Security

FINAL SEMESTER PROJECT

Phase 1 — Project Proposal and Documentation

PhishGuard: An Open-Source Phishing URL Detection, Reporting and Cyber-Awareness Platform

Submitted in partial fulfilment of the requirements of the course

SE-494 — Open Source Software Development

Student Name	Ahmed Abdul Wasay
Roll Number	F2024408023
Program	Bachelor of Science in Cyber Security
Course	SE-494 Open Source Software Development
Section	Y9

Submission Date: **6th June, 2026**

Abstract

Phishing remains one of the most prevalent and damaging cyber threats faced by individuals and organizations across the globe. Despite continuous advancements in security tooling, end-users frequently fall victim to fraudulent URLs that imitate legitimate websites, resulting in credential theft, financial loss and reputational damage. This project proposes the design, development and deployment of PhishGuard, an open-source full-stack web application that empowers users to verify suspicious URLs in real time, report newly discovered phishing campaigns, and learn about safe online practices through interactive awareness modules. The system is built using FastAPI as the backend framework, MongoDB as the document-oriented database, and HTML/CSS with light JavaScript for the user interface. PhishGuard exposes a RESTful API surface comprising more than ten endpoints supporting complete CRUD operations, role-based access for normal users and administrators, JSON Web Token (JWT) based authentication and rule-based heuristic scoring for phishing detection. The platform will be deployed using Render for the backend, Vercel for the static frontend, and MongoDB Atlas for managed database hosting. This document constitutes Phase 1 of the final semester project and presents the problem statement, objectives, target users, feature catalogue, technology stack, database schema, API specification, deployment plan and expected outcomes in accordance with IEEE academic documentation standards.

Keywords — Phishing Detection, Cyber Security, FastAPI, MongoDB, REST API, Full-Stack Web Application, JWT Authentication, Open Source, Threat Intelligence, IEEE.

Table of Contents

I. Introduction
II. Problem Statement
III. Project Objectives
IV. Target Users
V. Main Features
VI. Frontend Pages
VII. Backend API Endpoints
VIII. Database Choice and Schema Design
IX. Technology Stack
X. Deployment Plan
XI. Expected Output and Deliverables
XII. Risks and Mitigation
XIII. Conclusion
References

I. INTRODUCTION

1.1 Background

The rapid digitization of personal, financial and governmental services has dramatically expanded the attack surface available to malicious actors. According to publicly available industry reports, phishing accounts for the majority of social-engineering attacks observed worldwide and continues to evolve in sophistication. Attackers create deceptive websites that closely resemble legitimate platforms — banks, social networks, learning management systems and online retailers — and trick users into disclosing sensitive information such as passwords, payment details or one-time passcodes.

1.2 Motivation

While enterprise security teams employ advanced filtering and detection systems, ordinary Internet users, students and small businesses rarely have access to such tools. Free, lightweight and easy-to-use awareness platforms that combine detection with education are scarce, particularly within the open-source ecosystem. As a student of Cyber Security, the author was motivated to develop an accessible solution that not only detects suspicious URLs but also empowers users with knowledge to recognise and resist phishing attempts in their daily online interactions.

1.3 Scope of the Project

PhishGuard is scoped as an academic full-stack web application that demonstrates competence in modern open-source technologies. It provides URL submission and scoring, community phishing reports, administrative threat management, learning modules with quizzes, and user account management. The system is designed to be deployed publicly so that any user with an Internet connection can register, scan URLs and contribute to a shared threat intelligence repository. The scope is intentionally bounded so that the project can be completed, deployed and demonstrated within the timeline defined by the course instructor.

1.4 Document Organization

The remainder of this document is organised as follows. Section II presents the problem statement and motivation. Section III outlines the objectives. Section IV identifies the target users and personas. Section V enumerates the main features. Section VI catalogues the frontend pages. Section VII specifies the backend REST API. Section VIII justifies the database choice and presents the schema. Section IX details the complete technology stack. Section X describes the deployment plan. Section XI lists the expected outputs and deliverables. Section XII discusses risk and mitigation. Section XIII concludes the document and is followed by references.

II. PROBLEM STATEMENT

Phishing attacks exploit human trust rather than technological vulnerabilities, making them exceptionally difficult to prevent through purely technical means. End users frequently lack the time, expertise or tools required to evaluate the legitimacy of an unfamiliar URL before clicking it. Existing commercial threat-intelligence services are either prohibitively

expensive for individuals, restricted by API quotas, or offered only as opaque browser plug-ins that provide little educational value. Furthermore, there is no widely adopted open-source platform that allows users to (i) submit a URL and receive an immediate, explainable risk assessment, (ii) report newly observed phishing sites to a shared repository, and (iii) learn about phishing through structured awareness modules — all within a single application. The absence of such a unified, open and educational solution constitutes the core problem that PhishGuard seeks to address.

III. PROJECT OBJECTIVES

The principal objectives of the proposed PhishGuard project are enumerated below:

1. To design and develop a full-stack web application that can detect, record and report phishing URLs in a user-friendly manner.
2. To implement a RESTful backend using FastAPI exposing at least ten well-documented API endpoints supporting complete CRUD operations.
3. To integrate a NoSQL MongoDB database with clearly defined collections, indexes and validation rules.
4. To deliver a clean, accessible and responsive frontend consisting of at least five HTML/CSS pages aligned with modern usability principles.
5. To implement secure user authentication and authorization using hashed credentials and JSON Web Tokens (JWT).
6. To incorporate a rule-based heuristic phishing detection engine that produces an explainable risk score for any submitted URL.
7. To provide an administrative dashboard for managing users, reports and learning content.
8. To deploy the application publicly using free-tier open-source-friendly cloud platforms.
9. To document the entire engineering process following IEEE academic documentation standards.
10. To demonstrate the project, defend design decisions during the viva and contribute the source code as an open-source repository on GitHub.

IV. TARGET USERS

PhishGuard is designed to serve a diverse user community. The following personas describe its primary stakeholders.

- **General Internet Users.** Individuals who wish to verify whether a URL received via email, SMS or social media is safe before visiting it. They benefit from immediate detection results and educational tips.
- **Students and Educators.** Cyber security and computing students who use PhishGuard as a learning aid; instructors who can reference the platform during

awareness sessions and recommend the learning modules to their classes.

- **Small Business Owners.** Operators of small businesses without a dedicated security team who need a quick, free tool to check URLs and train their staff against phishing.
- **Security Researchers / Reporters.** Community members who actively identify and report phishing campaigns; their reports enrich the shared threat repository and benefit other users.
- **System Administrators.** Privileged users responsible for moderating user-submitted reports, curating learning content, managing user accounts and monitoring platform health.

V. MAIN FEATURES

The application provides the following ten primary features:

- **F1. User Registration and Login** Users can create accounts using an email and a strong password. Passwords are hashed using bcrypt prior to storage. Successful authentication returns a JWT used to authorise subsequent requests.
- **F2. URL Scanning Engine** Authenticated users submit a URL through a simple form. The backend extracts lexical, host-based and structural features (such as URL length, presence of IP address, number of subdomains, suspicious keywords, use of URL shorteners and the absence of HTTPS) and computes a transparent risk score classified as Safe, Suspicious or Malicious.
- **F3. Scan History** Each user maintains a personal history of previously scanned URLs together with their scores and timestamps, enabling auditability and trend tracking.
- **F4. Community Phishing Reports** Users can submit detailed reports about phishing sites they have encountered, including the URL, target organisation, evidence and a short description. Reports become part of the shared repository once approved by an administrator.
- **F5. Cyber Awareness Learning Modules** Structured educational content explains how phishing works, common indicators, safe email and browsing habits, and recovery steps after compromise. Short quizzes reinforce key learning outcomes.
- **F6. Admin Dashboard** Administrators can view platform statistics, moderate community reports, create or edit learning modules and manage user accounts. Access is restricted via role-based authorization.
- **F7. Search and Filter** Users can search the public phishing repository by domain, target brand or report date, and filter scan history by risk level.
- **F8. Profile Management** Users can update their display name, change their password and view their contribution statistics, including the number of scans performed and reports submitted.
- **F9. Responsive User Interface** The HTML/CSS frontend is designed with a mobile-first responsive layout, ensuring usability on phones, tablets and desktops.

- **F10. Public REST API** All backend functionality is exposed through documented REST endpoints, automatically rendered as interactive Swagger UI by FastAPI, enabling external integrations and transparent evaluation.

VI. FRONTEND PAGES

The frontend consists of the following seven HTML/CSS pages, exceeding the course minimum of five pages.

- **P1. index.html (Home / Landing Page)** Introduces PhishGuard, summarises its value proposition, provides quick-scan access for guests and offers navigation to login, registration and learning sections.
- **P2. login.html / register.html (Authentication Pages)** Provide forms for new-user registration and existing-user login, with client-side validation and inline error messaging.
- **P3. dashboard.html (User Dashboard)** Acts as the central authenticated workspace. Displays the URL scanning form, recent scan history, contribution statistics and quick links to learning modules.
- **P4. report.html (Submit and Browse Phishing Reports)** Allows users to submit new phishing reports and browse a searchable, filterable list of publicly approved reports.
- **P5. learn.html (Cyber Awareness Modules)** Hosts categorised learning content with embedded short quizzes and a progress indicator.
- **P6. admin.html (Administrator Console)** Restricted page enabling administrators to moderate reports, manage users, edit learning modules and view platform analytics.
- **P7. profile.html (User Profile and Settings)** Allows users to view and update their personal information, change passwords and review their contribution history.

VII. BACKEND API ENDPOINTS

The backend exposes a RESTful API consisting of the sixteen endpoints documented in Table I, exceeding the course minimum of ten endpoints.

Table I. PhishGuard REST API Endpoint Specification

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user account.
POST	/api/auth/login	Authenticate a user and return a JWT.
GET	/api/auth/me	Retrieve the currently authenticated user's profile.
PUT	/api/users/me	Update the authenticated user's profile information.
POST	/api/scan	Submit a URL for phishing risk analysis.
GET	/api/scan/history	List the authenticated user's previous URL scans.
GET	/api/scan/{scan_id}	Retrieve the full details of a specific scan.

Method	Endpoint	Purpose
DELETE	/api/scan/{scan_id}	Delete a specific scan from the user's history.
POST	/api/reports	Submit a new community phishing report.
GET	/api/reports	List approved community phishing reports with optional filters.
GET	/api/reports/{report_id}	Retrieve a specific phishing report by identifier.
PUT	/api/reports/{report_id}	Update the status of a phishing report (admin only).
DELETE	/api/reports/{report_id}	Delete a phishing report (admin only).
GET	/api/modules	List all cyber awareness learning modules.
POST	/api/modules	Create a new learning module (admin only).
GET	/api/admin/stats	Retrieve aggregated platform statistics (admin only).

VIII. DATABASE CHOICE AND SCHEMA DESIGN

8.1 Database Choice and Justification

After comparative evaluation of the two database options permitted by the course (PostgreSQL and MongoDB), MongoDB was selected for PhishGuard. The justification is threefold. First, the data handled by the system — phishing reports, scan results and learning modules — is naturally hierarchical and semi-structured, containing variable evidence fields and nested feature vectors that fit a document model more elegantly than a normalised relational schema. Second, MongoDB integrates seamlessly with FastAPI through the asynchronous Motor driver, simplifying I/O and supporting high concurrency. Third, MongoDB Atlas provides a free managed tier that satisfies the deployment and hosting requirement of the course.

8.2 Collection Schemas

The database comprises the following five MongoDB collections.

Collection: users

- **_id** — ObjectId — primary identifier
- **email** — String — unique, indexed
- **password_hash** — String — bcrypt hash
- **display_name** — String
- **role** — String — "user" or "admin"
- **created_at** — ISODate
- **updated_at** — ISODate

Collection: scans

- **_id** — ObjectId
- **user_id** — ObjectId — reference to users

- **url** — String
- **features** — Object — extracted lexical and host features
- **risk_score** — Number (0–100)
- **verdict** — String — Safe / Suspicious / Malicious
- **created_at** — ISODate

Collection: reports

- **_id** — ObjectId
- **submitted_by** — ObjectId — reference to users
- **url** — String
- **target_brand** — String
- **description** — String
- **evidence** — Array of Strings — URLs or notes
- **status** — String — pending / approved / rejected
- **created_at** — ISODate
- **reviewed_at** — ISODate

Collection: modules

- **_id** — ObjectId
- **title** — String
- **category** — String
- **content** — String — Markdown or HTML
- **quiz** — Array of Objects — questions and answers
- **created_by** — ObjectId — reference to users (admin)
- **created_at** — ISODate

Collection: audit_logs

- **_id** — ObjectId
- **actor_id** — ObjectId
- **action** — String — e.g., REPORT_APPROVED
- **target_id** — ObjectId
- **timestamp** — ISODate

IX. TECHNOLOGY STACK

Table II summarises the complete technology stack adopted for the project. Every component is open-source or offers a permissive free tier suitable for academic use.

Component	Technology / Tool
Backend Framework	FastAPI (Python 3.11)
Asynchronous Server	Uvicorn
Database	MongoDB (managed via MongoDB Atlas)
Database Driver	Motor (asynchronous PyMongo)
Authentication	OAuth2 password flow with JSON Web Tokens (PyJWT) and bcrypt password.
Data Validation	Pydantic v2 models
Frontend Markup	HTML5
Frontend Styling	CSS3 with custom design tokens and responsive layouts
Frontend Interactivity	Vanilla JavaScript (Fetch API) — optional progressive enhancement.
API Documentation	Auto-generated Swagger UI / OpenAPI provided by FastAPI
Version Control	Git and GitHub (public open-source repository)
Deployment — Backend	Render (free tier)
Deployment — Frontend	Vercel (free tier)
Deployment — Database	MongoDB Atlas (free shared cluster)
Testing Tools	Pytest and HTTPX for backend; manual end-to-end testing for frontend.
Documentation	Microsoft Word / LaTeX following IEEE format

X. DEPLOYMENT PLAN

The deployment strategy ensures the application is publicly accessible, free to operate and easy to evaluate by the instructor.

Backend Deployment. The FastAPI application is containerised through a simple requirements.txt and Procfile and deployed to Render. Render automatically provisions an HTTPS endpoint of the form <https://phishguard-api.onrender.com> and rebuilds the service on each push to the main branch of the GitHub repository, enabling continuous deployment.

Frontend Deployment. The static HTML, CSS and JavaScript files are deployed to Vercel, which provides a global CDN, automatic HTTPS and instant rebuilds on Git commits. The frontend communicates with the backend exclusively through the public API URL configured via an environment variable.

Database Hosting. A free MongoDB Atlas shared cluster hosts the database. Network access is restricted to the deployed backend IP range, and a dedicated application user with least-privilege credentials is used.

Security and Configuration. All secrets — JWT signing key, database URI and admin credentials — are stored as environment variables on the hosting platforms and never committed to source control. HTTPS is enforced end-to-end and CORS is restricted to the

deployed frontend domain.

XI. EXPECTED OUTPUT AND DELIVERABLES

Upon successful completion of all three project phases the following deliverables will be produced:

- A live, publicly accessible PhishGuard web application with a registered domain provided by the hosting platforms.
- A public GitHub repository containing the complete, well-documented source code under an open-source licence.
- An interactive Swagger UI exposing every backend endpoint, enabling transparent evaluation.
- A populated MongoDB Atlas database containing seeded administrator accounts, sample phishing reports and at least three learning modules.
- A README file with setup instructions, environment variable templates and demo credentials.
- Screenshots demonstrating each frontend page, the admin dashboard, the API documentation and the database collections — to be included with the Phase 2 / Phase 3 submission.
- This IEEE-style Phase 1 documentation in PDF format.

XII. RISKS AND MITIGATION

Software projects inevitably face risks. The principal risks identified for PhishGuard, together with their mitigation strategies, are summarised below.

- **Limited Time Budget.** Plan and follow a weekly milestone schedule; prioritise core features (authentication, scanning, reports) before optional enhancements.
- **Free-Tier Hosting Constraints.** Use cold-start aware design, keep payloads small, and document any tier-specific limitations in the README.
- **Phishing Heuristics False Positives.** Provide an explainable score breakdown so users understand why a URL was flagged, and allow administrators to whitelist false positives.
- **Security of Stored Credentials.** Hash all passwords with bcrypt, sign JWTs with a strong secret, enforce HTTPS and follow OWASP Top 10 mitigation practices.
- **Scope Creep.** Freeze the feature list defined in Section V; any additional ideas will be deferred to a future post-submission roadmap.

XIII. CONCLUSION

This Phase 1 document has presented the complete plan for PhishGuard, an open-source full-stack web application that combines phishing URL detection, community reporting and structured cyber-awareness education in a single platform. The project aligns with the

learning outcomes of the Open Source Software Development course (SE-494) by integrating modern open-source technologies — FastAPI, MongoDB, HTML, CSS — and by emphasising deployable, real-world software engineering practice. The proposed system satisfies and exceeds the minimum requirements stipulated by the course instructor, comprising seven frontend pages, sixteen documented REST API endpoints, five MongoDB collections with full CRUD support, role-based authentication, public deployment and comprehensive academic documentation. Upon approval of this proposal, the author will proceed to Phase 2 — the implementation and deployment of the working system — and subsequently to Phase 3, where the final artefacts will be presented during the viva.

REFERENCES

- [1] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report," APWG, 2024. [Online]. Available: <https://www.apwg.org/trendsreports/>
- [2] FastAPI, "FastAPI Framework Documentation," FastAPI, 2025. [Online]. Available: <https://fastapi.tiangolo.com/>
- [3] MongoDB Inc., "MongoDB Manual," MongoDB, 2025. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [4] Open Web Application Security Project (OWASP), "OWASP Top 10 - 2021," OWASP Foundation, 2021. [Online]. Available: <https://owasp.org/Top10/>
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [6] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," Internet Engineering Task Force, RFC 3986, January 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3986>
- [7] S. Garera, N. Provos, M. Chew, and A. D. Rubin, "A framework for detection and measurement of phishing attacks," in Proceedings of the 2007 ACM Workshop on Recurring Malcode (WORM '07), 2007, pp. 1-8.
- [8] Render Inc., "Render Deployment Documentation," Render, 2025. [Online]. Available: <https://render.com/docs>
- [9] Vercel Inc., "Vercel Documentation," Vercel, 2025. [Online]. Available: <https://vercel.com/docs>
- [10] IEEE, "IEEE Editorial Style Manual for Authors," IEEE, 2024. [Online]. Available: <https://journals.ieeeauthorcenter.ieee.org/>